EventHub AIOps

Multi-agent incident investigation for the banking domain

Evidence-backed root cause and runbook-cited actions, with a human in control

4 LLM agents | LangGraph workflow | MCP tools | RAG over runbooks | LangFuse + DeepEval | Human feedback loop

Built and tested in seven phases · 427 automated tests · real-model evaluation

AI Capstone Project · Demo · version 4
Submitted by:
- Saravanavel Murugan
- Abhijeet Khopkar

PROBLEM CONTEXT

Incidents in banking are slow to diagnose

The cost

High-severity incidents bring long time to acknowledge (MTTA) and time to resolve (MTTR).

The result: financial penalties, regulatory non-compliance and lost customer trust.

Why it is hard

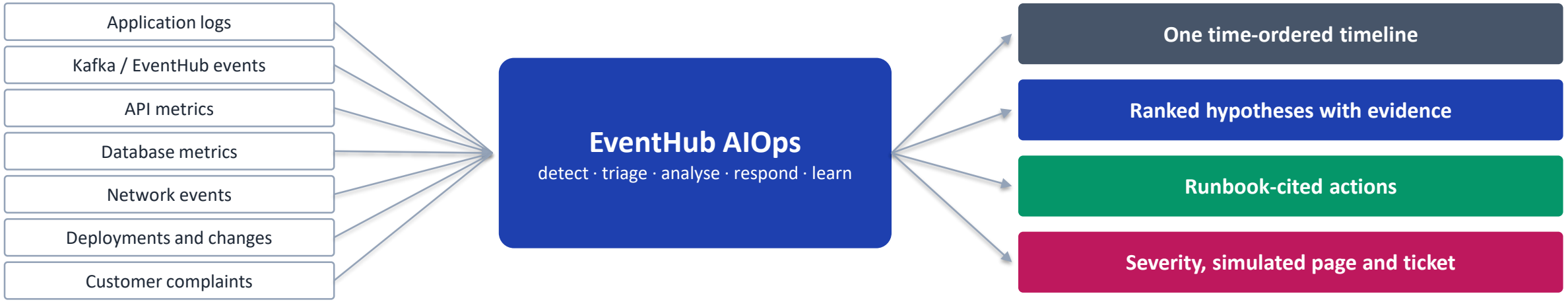
Evidence sits in silos: logs, Kafka events, API, database and network metrics, deployments and customer complaints.

Engineers rebuild the timeline by hand. Failures on EventHub, the bank's Kafka platform, show up first as symptoms in other services.

What we built

One AI system that merges every source into a single timeline and returns ranked root causes with evidence, and runbook-cited actions.

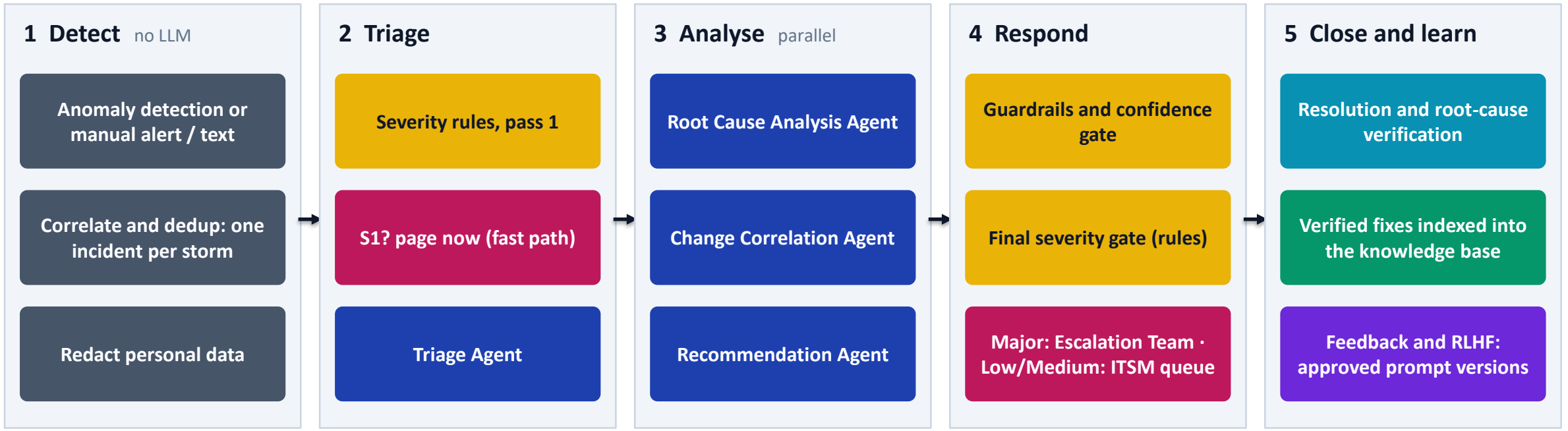
Rules page the right team within seconds; a human reviews and approves every recommendation.



Five synthetic scenarios: bad release · Kafka consumer lag · core DB saturation · expired TLS certificate · silent duplicate debits

TOP-LEVEL ARCHITECTURE

Five stages: agents analyse, rules decide, humans approve



Shared platform

incident-tools MCP server
13 read-only tools: telemetry, EventHub, complaints, dependencies, knowledge search. No paging or ticketing tools.

FAISS knowledge base
runbooks, postmortems, verified fixes

Incident State Store
lifecycle record

Simulated outbox
pages, tickets, chat, email

LLM agents analyse · deterministic rules decide severity and dispatch · humans approve every recommendation

FUNCTIONALITY OF EACH AGENT

Four LLM agents, each with a narrow job

Triage Agent
What kind of incident is this?

DOES
Classifies into 10 issue classes, proposes a severity, lists affected services (origin, impacted, downstream).

USES
Anomaly events, the redacted report, platform and complaint summaries.

SAFEGUARD
Its severity is advisory: the rules engine decides. Evidence IDs are checked against real records.

Root Cause Analysis Agent
Why did it happen?

DOES
Builds the timeline and ranked root-cause hypotheses with confidence, supporting and missing evidence.

USES
Read-only tools over MCP: logs, Kafka, API, DB, network, complaints, dependencies. Two rounds, up to 8 calls, run in parallel.

SAFEGUARD
Tool output is treated as untrusted data (injection defence); every cited ID must exist.

Change Correlation Agent
Did a change cause it?

DOES
Decides which recent releases, config, infra, ACL or schema changes contributed, and links them to a hypothesis.

USES
Facts computed in code: changes in a 3-hour look-back, minutes before onset, on the dependency path or not.

SAFEGUARD
Strict red-herring rules; code supplies the facts, the LLM only ranks.

Recommendation Agent
What should we do?

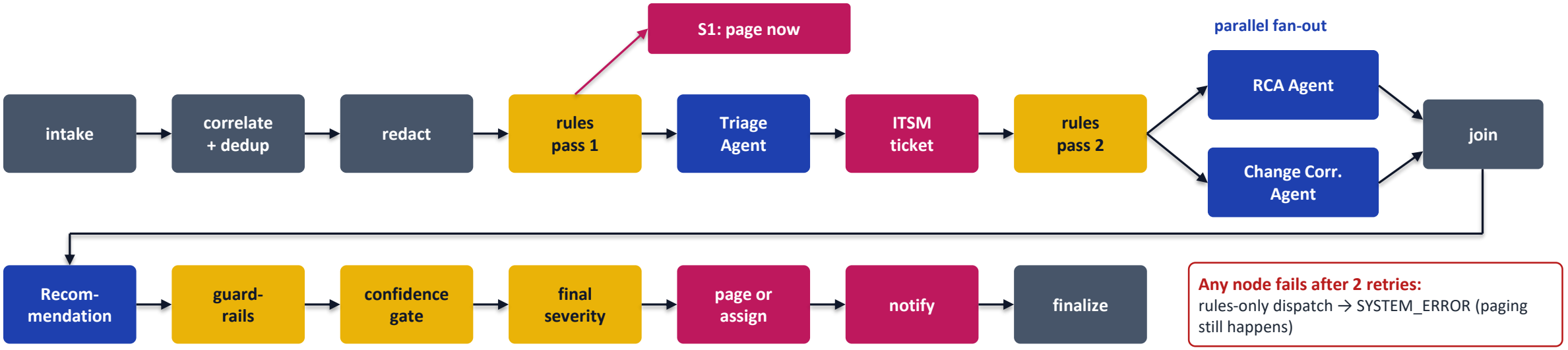
DOES
Proposes ordered fix steps, a stakeholder summary and regulatory notes.

USES
Targeted retrieval from FAISS: runbooks, postmortems, verified past fixes.

SAFEGUARD
Action policy removes denied actions and flags high-risk ones; abstains with INSUFFICIENT_EVIDENCE when evidence is weak.

Around the agents, no LLM: anomaly detector · alert correlation and dedup · severity rules engine (two passes and a re-score, raise-only) · confidence and severity gates · notification service · output guardrails (action policy, grounding, PII, tone)

LangGraph orchestrates, LangChain talks to models and tools



LangGraph

- **StateGraph** over a typed state; reducers merge writes from parallel nodes safely
- **22 nodes, conditional edges** route the S1 fast path, gates and failures
- **Fan-out and join:** RCA and Change Correlation run in parallel
- **Node wrapper:** 2 retries with backoff, then the rules-only fallback
- **graph.stream()** feeds live node-by-node progress to the UI

LangChain

- **ChatOpenAI:** gpt-4o-mini, temperature 0.2, 1,500-token output cap
- **Structured output** through function calling into Pydantic schemas (never strict JSON-schema mode)
- **bind_tools** with tools loaded from the MCP server (langchain-mcp-adapters)
- **Embeddings + FAISS** retrieval; text splitters chunk the knowledge base
- **Callbacks** send every generation to LangFuse; per-run cost cap of $0.15

TECH STACK

Built on current open-source releases, Python 3.10

Orchestration	**LangGraph 1.2.13** StateGraph, parallel nodes, streaming
LLM framework	**LangChain core 1.6.6 · langchain-openai 1.6.7** chat models, tools, structured output
Models	**OpenAI gpt-4o-mini · text-embedding-3-small** through the Vocareum OpenAI gateway
Tool protocol	**MCP 1.30 (FastMCP, stdio) · langchain-mcp-adapters 0.3.2** 13 read-only tools
Retrieval	**FAISS 1.15 · LangChain text splitters · pdfplumber** 28 runbooks, postmortems, policies
Evaluation	**DeepEval 4.2.8** LLM-as-judge metrics on the golden set
Observability	**LangFuse Cloud 4.17 · JSON logs** traces, scores, datasets, audit logs
User interface	**Streamlit 1.65 (multipage)** eight pages, from investigation to adaptation
Data and config	**Pydantic 2.13 · pydantic-settings · pandas · PyYAML** typed schemas, versioned rules
Quality and delivery	**pytest 9.1 · GitHub · Vocareum lab** 427 offline tests with a scripted LLM, plus real-model checks

Measured against a golden dataset, traced end to end

LangFuse Cloud: observe and track

- **One trace per investigation:** spans for every node and agent, LLM generations with tokens and cost (about 75 observations per run)
- **Session and tags:** run ID, scenario and run mode
- **Scores per run:** DeepEval metrics and the reward score
- **Datasets:** the golden set mirrored for tracking over time; latency and cost per stage

DeepEval: score the answers

- **LLM-as-judge** with a custom judge that uses function calling
- **GEval:** does the top hypothesis match the ground-truth root cause?
- **Faithfulness and Hallucination:** are the claims supported by the cited evidence?
- **Contextual recall:** did retrieval find the expected runbook?
- **Evaluation run mode:** the full graph runs, but nothing is dispatched

Golden dataset: 25 cases, growing with SME promotions

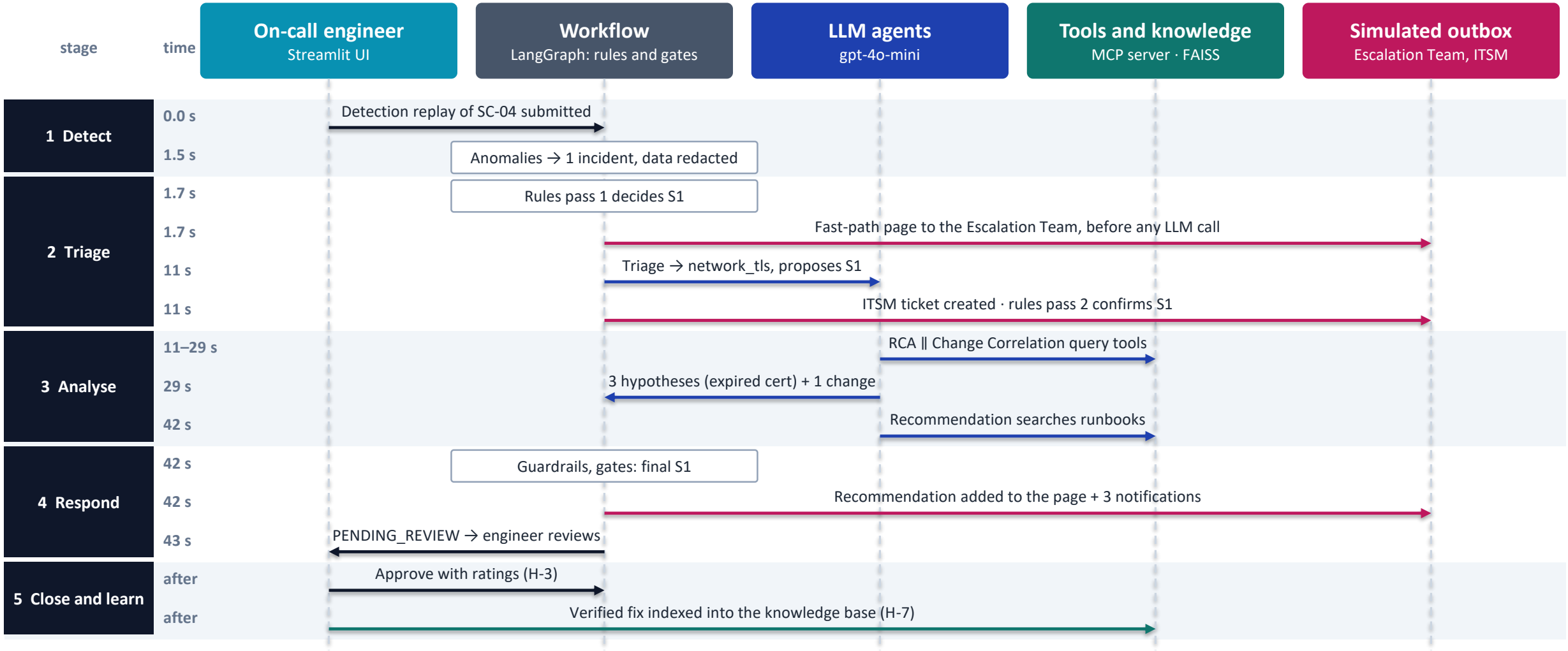
5 scenarios × 5 variants: alert or replay, free text, narrow window, wide noisy window, and a missing source (the right answer is to abstain). Evaluation runs use their own incident store and dispatch nothing. Plus 4 fixtures: tool failure, LLM down, alert storm, prompt injection.

Metric	Target	Measured
Root cause, top-1 / top-3	≥ 60% / ≥ 80%	0.75–1.0
Faithfulness	≥ 0.80	0.66–0.90
Hallucination rate	≤ 10%	0%
Retrieval recall@5	≥ 0.85	1.0
Citation validity	≥ 95%	100%
Severity, class, routing	≥ 70–100%	100%
Critical page latency	≤ 10 s	0.1–0.2 s
Fixtures: failure, LLM down, dedup, injection	100%	4 / 4 pass
Time per investigation	p50 ≤ 30 s	≈ 45–60 s
Cost per investigation	≤ $0.15 cap	≈ $0.004

Measured with gpt-4o-mini through the Vocareum gateway: SC-04 golden cases with the DeepEval judge, and the fixture checks. Fill in the full 25-case run before the demo.

SEQUENCE FLOW

One scenario end to end, stage by stage: expired TLS certificate (SC-04)



Times from a real run through the UI: 43 s in total, about $0.004. The S1 page went out at 1.7 s, before any LLM had answered. Stage 5 happens after the run.

SAFETY

Five layers of guardrails, from input to learning

1 Input

- Personal data redacted before any LLM sees it (consistent pseudonyms)
- Prompt-injection detector; tool and complaint text passed as untrusted data
- Kill switch: LLM_ENABLED=false runs on rules only

2 Tools

- Agents get read-only tools, only through the MCP server
- No page, ticket or notify tool exists on that server
- Other tool calls are blocked and logged
- 8 tool calls per RCA; cost cap $0.15 per run

3 Output

- Schema check on every agent answer
- Grounding: claims must cite real evidence; ungrounded ones removed
- Action policy: denied actions removed, high-risk ones flagged
- PII re-check and tone check

4 Dispatch

- Severity and paging decided by rules, never by an LLM
- One incident per alert storm; one page per 15 minutes
- Every page, ticket and message is simulated
- High-risk actions need a two-step human confirmation

5 Learning

- Only prompt guidelines, few-shot examples and retrieval aliases can change
- Every change approved by the SME or Evaluation Engineer
- Two reviewers per preference; at most 30% from one reviewer
- Golden-set check before and after; revert on regression

Tested, not just designed: PII, injection, action-policy and tool-access cases run in the automated tests, and the injection fixture passes against the real model.

Every human decision becomes a learning signal

1 Collect

- Review: approve, edit or reject
- Ratings 1–5 on four dimensions
- Re-classification to Major
- "Needs human RCA" hand-off
- Fix outcome
- Root-cause verification
- Pairwise A/B (blind)

2 Store and check

- Append-only preference store
- PII re-checked on every record
- Duplicates detected
- Two reviewers must agree; SME breaks ties
- At most 30% of signals from one reviewer

3 Learn

- Edits and rejections → SME review queue → golden dataset
- Reward score per prompt version: ratings 0.35, win rate 0.30, approvals 0.15, fix outcome 0.20 (min. 10 signals)
- DPO export of agreed pairs

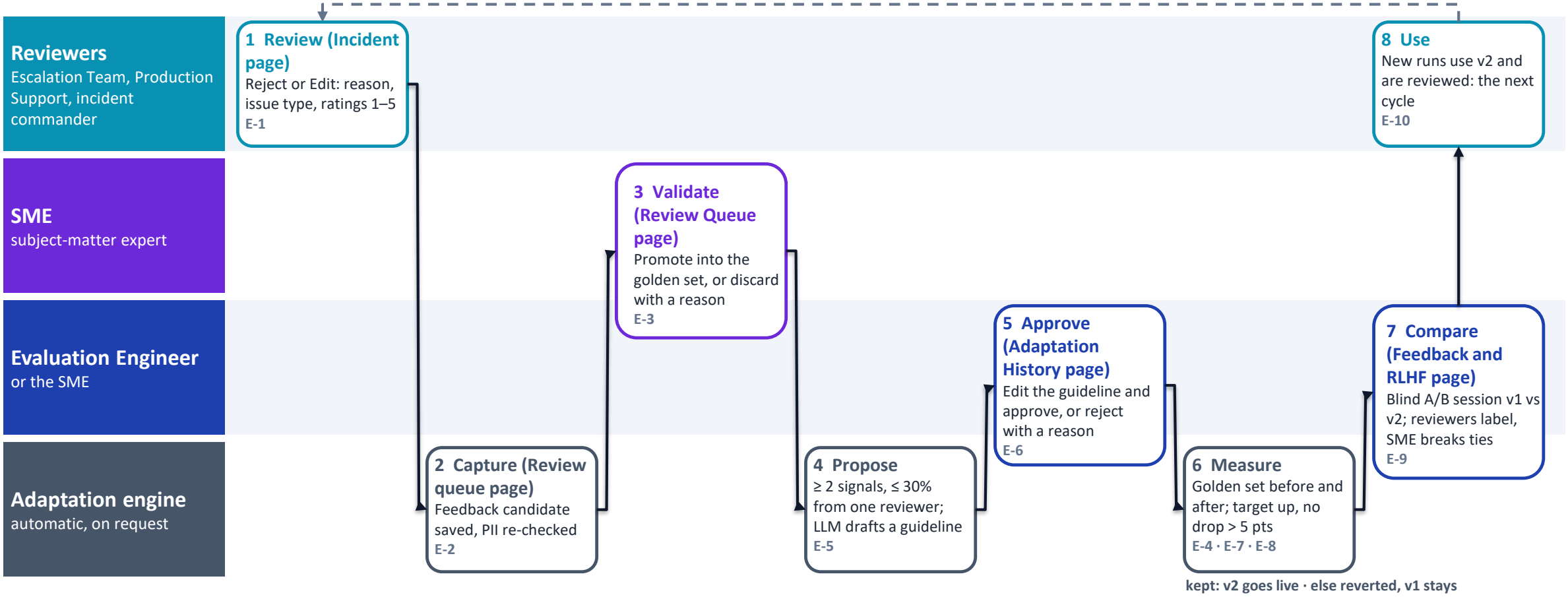
4 Adapt

- Adaptation engine proposes a prompt rule, few-shot example or retrieval alias
- SME approves
- Before/after run on the golden set and held-out pairs
- Kept if better with no regression, otherwise reverted

new prompt version goes live for the LLM agents, then is reviewed again

Provable from the logs: the evidence report follows one adaptation from the first review (E-1) to better-reviewed new runs (E-10) and marks any missing step. **Also:** verified root causes are indexed into the knowledge base. Adaptation never edits code, severity rules or the action policy.

Who does what, from one human review to a new prompt version



Guardrails: nothing changes without the SME's validation, a human approval and a before/after check on the golden set. Adaptation may only add prompt guidelines, few-shot examples or retrieval aliases, never severity rules, thresholds or the model.
Provable from the logs: the evidence report finds each step, E-1 to E-10, in interactions.log and eval.log.

LIVE DEMO

Demo walkthrough: one incident, then the learning loop

1 Investigate
On-call

Replay SC-04. Watch the nodes; the S1 page goes out in seconds, before any LLM call.

2 Incident
Escalation

Open evidence behind each claim; check the actions; Edit with a reason and ratings.

3 Outbox
Everyone

One simulated page, one ticket, three notifications. Nothing real was sent.

4 Resolve and Verify
Resolver, SME

Record the fix; the SME confirms it, and it is indexed for next time.

5 Review Queue
SME

Promote the edited case into the golden dataset (version goes up).

6 Feedback and RLHF
Two reviewers

Label A/B pairs blind; see agreement, win rate and reward score.

7 Adaptation History
SME

Scan, approve the drafted guideline, see before/after metrics: kept or reverted.

8 Evaluation
Eval. Engineer

Metrics against targets, LangFuse traces, and the E-1 to E-10 evidence report.

Before the demo: run the full golden set (about 12 minutes) and one adaptation (about 25 minutes), so their results are ready to show; run step 1 live.

Eight Streamlit pages, one for each job

Page	Who	What happens there	Human step
Investigate	On-call engineer	Replay, alert JSON or free text; live node-by-node progress; early result with the fast-path page	
Incident	Escalation, Production Support, incident commander	Hypotheses with one-click evidence, changes, actions, timeline; review with ratings; acknowledge; re-classify	H-1 to H-5
Outbox	Everyone	Simulated pages, tickets and notifications; reset demo	
Resolve and Verify	Resolver, SME	Actual root cause and fix outcome; verified fixes indexed for later citation	H-6, H-7
Review Queue	SME	Promote edited or rejected outputs into the golden dataset, or discard	H-8
Feedback and RLHF	Reviewers, Evaluation Engineer, SME	Blind A/B labels, SME tie-break, reward scores, DPO export	H-9
Adaptation History	SME, Evaluation Engineer	Approve and apply changes; before/after metrics; evidence report; rules review	H-10
Evaluation	Evaluation Engineer	Golden set and fixtures; metrics against targets; LangFuse traces	

No sign-in in the prototype: the user picks a role and reviewer ID in the sidebar, and both are recorded with every decision. Status always shows as a label, never colour alone.

THANK YOU